程序设计原理

多维数组

任课教师： 喻　梅
于瑞国
王建荣
李雪威
赵满坤

Exercise

Sample Input

```
47
S 10
S 11
S 12
S 13
H 1
H 2
S 6
S 7
S 8
S 9
H 6
H 8
H 9
H 10
H 11
H 4
H 5
S 2
S 3
S 4
S 5
H 12
H 13
C 1
C 2
D 1
D 2
D 3
D 4
D 5
D 6
D 7
C 3
C 4
C 5
C 6
C 7
C 8
C 9
C 10
C 11
C 13
D 9
D 10
D 11
D 12
D 13
```

Sample Output

```
S 1
H 3
H 7
C 12
D 8
```

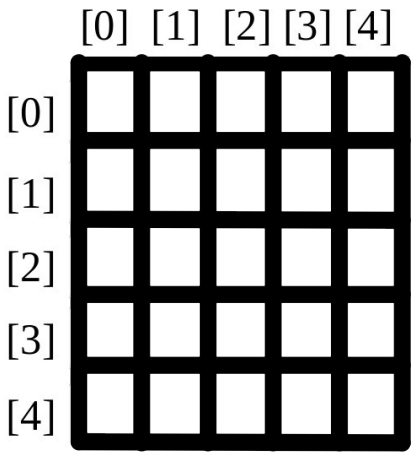
Two-dimensional Arrays

```
// Declare array ref var
```

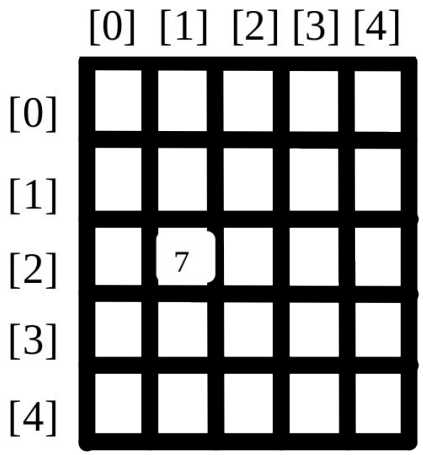
elementType arrayName[rowSize][columnSize];

int matrix[5][5];

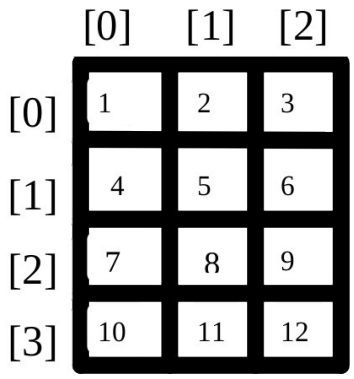
Two-dimensional Array Illustration




```
int matrix[5][5];
```




```
matrix[2][1] = 7;
```




```
int array[4][3] =
  {1, 2, 3},
  {4, 5, 6},
  {7, 8, 9},
  {10, 11, 12}
};
```

int array[][3] = { {1,2,3}, {4,5,6} };

多维数组**必须**从**右往左**保证后续维度已知

Declaring, Creating, and Initializing Using Shorthand Notations

You can also use an array initializer to declare, create and initialize a two-dimensional array. For example,

```
int array[4][3] =
{
  {1, 2, 3},
  {4, 5, 6},
  {7, 8, 9},
  {10, 11, 12}
};
```

(a)

Equivalent

```
int array[4][3];
array[0][0] = 1; array[0][1] = 2; array[0][2] = 3;
array[1][0] = 4; array[1][1] = 5; array[1][2] = 6;
array[2][0] = 7; array[2][1] = 8; array[2][2] = 9;
array[3][0] = 10; array[3][1] = 11; array[3][2] = 12;
```

(b)

Printing Arrays

int matrix[5][5];

To print a two-dimensional array, you have to print each element in the array using a loop like the following:

```
for (int row = 0; row < rowSize; row++)
{
  for (int column = 0; column < columnSize; column++)
  {
    cout << matrix[row][column] << " ";
  }
  cout << endl;
}
```

Summing Elements by Column

For each column, use a variable named total to store its sum. Add each element in the column to total using a loop like this:

```
for (int column = 0; column < columnSize; column++)
{
  int total = 0;
  for (int row = 0; row < rowSize; row++)
    total += matrix[row][column];
  cout << "Sum for column " << column << " is " << total
<< endl;
}
```

Exercise

Sample Input

```
47
S 10
S 11
S 12
S 13
H 1
H 2
S 6
S 7
S 8
S 9
H 6
H 8
H 9
H 10
H 11
H 4
H 5
S 2
S 3
S 4
S 5
H 12
H 13
C 1
C 2
D 1
D 2
D 3
D 4
D 5
D 6
D 7
C 3
C 4
C 5
C 6
C 7
C 8
C 9
C 10
C 11
C 13
D 9
D 10
D 11
D 12
D 13
```

Sample Output

```
S 1
H 3
H 7
C 12
D 8
```

Exercise

```
int main(){
    int card[4][13]={0};
    int n;
    char suit;
    int rank;

    cin >> n;

    while (n>0){
        cin >> suit >> rank;
        switch(suit){
            case 'S':
                card[0][rank-1]=1;
                break;
            case 'H':
                card[1][rank-1]=1;
                break;
            case 'C':
                card[2][rank-1]=1;
                break;
            case 'D':
                card[3][rank-1]=1;
                break;
        }
```

```
    for(int i=0;i<4;i++){
        for(int j=0;j<13;j++){
            if(card[i][j]==0){
                switch(i){
                    case 0:
                        cout << 'S' <<' ' << j+1 << endl;
                        break;
                    case 1:
                        cout << 'H' <<' ' << j+1 << endl;
                        break;
                    case 2:
                        cout << 'C' <<' ' << j+1 << endl;
                        break;
                    case 3:
                        cout << 'D' <<' ' << j+1 << endl;
                        break;
                }
            }
        }
    }

    return 0;
}
```

Example: Grading Multiple-Choice Test

Objective: write a program that grades multiple-choice test.

Students' Answers to the Questions:

	0	1	2	3	4	5	6	7	8	9
Student 0	A	B	A	C	C	D	E	E	A	D
Student 1	D	B	A	B	C	A	E	E	A	D
Student 2	E	D	D	A	C	B	E	E	A	D
Student 3	C	B	A	E	D	C	E	E	A	D
Student 4	A	B	D	C	C	D	E	E	A	D
Student 5	B	B	E	C	C	D	E	E	A	D
Student 6	B	B	A	C	C	D	E	E	A	D
Student 7	E	B	E	C	C	D	E	E	A	D

Key to the Questions:

	0	1	2	3	4	5	6	7	8	9
Key	D	B	D	C	C	D	A	E	A	D

Example: Grading Multiple-Choice Test

```
#include <iostream>
using namespace std;
int main(){
  const int NUMBER_OF_STUDENTS = 8;
  const int NUMBER_OF_QUESTIONS = 10;

  // Students' answers to the questions
  char
answers[NUMBER_OF_STUDENTS][NUMBER_O
F_QUESTIONS] =
  {
    {'A', 'B', 'A', 'C', 'C', 'D', 'E', 'E', 'A', 'D'},
    {'D', 'B', 'A', 'B', 'C', 'A', 'E', 'E', 'A', 'D'},
    {'E', 'D', 'D', 'A', 'C', 'B', 'E', 'E', 'A', 'D'},
    {'C', 'B', 'A', 'E', 'D', 'C', 'E', 'E', 'A', 'D'},
    {'A', 'B', 'D', 'C', 'C', 'D', 'E', 'E', 'A', 'D'},
    {'B', 'B', 'E', 'C', 'C', 'D', 'E', 'E', 'A', 'D'},
    {'B', 'B', 'A', 'C', 'C', 'D', 'E', 'E', 'A', 'D'},
    {'E', 'B', 'E', 'C', 'C', 'D', 'E', 'E', 'A', 'D'}
  };
 // Key to the questions
  char keys[] = {'D', 'B', 'D', 'C', 'C', 'D', 'A', 'E', 'A',
'D'};

// Grade all answers
 for (int i = 0; i < NUMBER_OF_STUDENTS; i++)
 {
   // Grade one student
   int correctCount = 0;
   for (int j = 0; j < NUMBER_OF_QUESTIONS; j++)
   {
     if (answers[i][j] == keys[j])
       correctCount++;
   }

    cout << "Student " << i << "'s correct count is " <<
    correctCount << endl;
 }

  return 0;
}
```

Multidimensional Arrays

In the preceding section, you used a two-dimensional array to represent a matrix or a table. Occasionally, you will need to represent *n*-dimensional data structures. In C++, you can create *n*-dimensional arrays for any integer *n*.

The way to declare two-dimensional array can be generalized to declare *n*-dimensional array for *n* >= 3. For example, the following syntax declares a three-dimensional array scores.

```
double scores[10][5][2];
```

Multidimensional Arrays

```
int a[][2][3] = {
    { {1,2,3}, {4,5,6} },
    { {7,8,9}, {10,11,12} }
};
```

✔

```
int a[2][][3] = { { {1,2,3}, {4,5,6} } };  // × 错误
int a[][][3]  = { ... };                   // × 错误
```

Exercise

Official House

You manage 4 buildings, each of which has 3 floors, each of which consists of 10 rooms. Write a program which reads a sequence of tenant/leaver notices, and reports the number of tenants for each room.

For each notice, you are given four integers b, f, r and v which represent that v persons entered to room r of fth floor at building b. If v is negative, it means that −v persons left.

Assume that initially no person lives in the building.

Input

In the first line, the number of notices n is given. In the following n lines, a set of four integers b, f, r and v which represents ith notice is given in a line.

Output

For each building, print the information of 1st, 2nd and 3rd floor in this order. For each floor information, print the number of tenants of 1st, 2nd, .. and 10th room in this order. Print a single space character before the number of tenants. Print "####################" (20 '#') between buildings.

Exercise

Constraints

No incorrect building, floor and room numbers are given.

0 ≤ the number of tenants during the management ≤ 9

Sample Input

```
3
1 1 3 8
3 2 2 7
4 3 8 1
```

Sample Output

```
 0 0 8 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0
####################
 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0
####################
 0 0 0 0 0 0 0 0 0 0
 0 7 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0
####################
 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 1 0 0
```

Exercise

Matrix Vector Multiplication

Write a program which reads a n×m matrix A and a m×1 vector b, and prints their product Ab.

A column vector with *m* elements is represented by the following equation.
A n×m matrix with m column vectors, each of which consists of n elements, is represented by the following equation.

$$b = \begin{pmatrix} b_1 \\ b_2 \\ : \\ b_m \end{pmatrix} \qquad A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1m} \\ a_{21} & a_{22} & \dots & a_{2m} \\ : & : & : & : \\ a_{n1} & a_{n2} & \dots & a_{nm} \end{pmatrix}$$

i-th element of a *m×1* column vector *b* is represented by b_i *(i=1,2,...,m)*, and the element in *i-th* row and *j-th* column of a matrix A is represented by a_{ij} *(i=1,2,...,n, j=1,2,...,m).*

The product of a *n×m* matrix A and a *m×1* column vector b is a *n×1* column vector c, and c_i is obtained by the following formula:

$$c_i = \sum_{j=1}^{m} a_{ij} b_j = a_{i1} b_1 + a_{i2} b_2 + \ldots + a_{im} b_m$$

Exercise

Input

In the first line, two integers n
and m are given. In the following n lines, a_{ij} are given separated by a single space character. In the next m lines, b_i is given in a line.

Output

The output consists of n lines. Print c_i in a line.

Constraints

$1 \le n,m \le 100$ $0 \le bi,aij \le 1000$

Sample Input	Sample Output
3 4 1 2 0 1 0 3 0 1 4 1 1 0 1 2 3 0	5 6 9